

Multi-Source Candidate Data Transformer: Technical Design

Author: Anshita Verma

1. Pipeline Architecture

The pipeline is designed as a modular, deterministic ETL (Extract, Transform, Load) system. It strictly separates data ingestion from final output projection, ensuring the core engine remains immutable regardless of the runtime configuration.

- **Ingest:** Safely reads structured (JSON) and unstructured (REST API) inputs with fault tolerance.
- **Extract & Map:** Translates disparate source schemas into an internal, unified object format.
- **Normalize:** Scrubs individual fields via pure functions (e.g., stripping phone characters, date parsing).
- **Merge (Resolution):** Consolidates multiple source records into a single Canonical Profile using a confidence-based scoring algorithm.
- **Project & Validate:** Applies the runtime config to reshape the canonical record into the final schema.

2. Canonical Schema & Normalization

The system standardizes data into a strict internal canonical schema before projection.

- **Canonical Fields:** `candidate_id`, `full_name`, `emails[]`, `phones[]`, `location`, `links[]`, `headline`, `years_experience`, `skills[]`, `experience[]`, `education[]`, `overall_confidence`, `provenance[]`.
- **Normalization Rules:**
 - **Phones:** Stripped of all non-numeric characters and formatted to the standard **E.164** format (defaulting to the **+91** country code if absent).
 - **Dates:** Parsed and standardized to the **YYYY-MM** format for chronological consistency.
 - **Skills:** Standardized to lowercase, trimmed strings to allow for exact-match deduplication.

3. Merge Policy & Entity Resolution

Conflict resolution is handled mathematically rather than conditionally, ensuring deterministic outputs.

- **Confidence Assignment:** Each extraction module assigns a base confidence score to its output based on the reliability of the source (e.g., ATS JSON = `0.9` confidence; Public GitHub API = `0.7` confidence).
- **Single-Value Fields (Name, Location):** If sources conflict, the value from the source with the highest confidence score strictly overrides the others.
- **Array Fields (Emails, Phones, Skills):** Arrays are deeply merged and deduplicated. Unique values from lower-confidence sources are appended, preserving maximum candidate contact vectors without data loss.
- **Provenance:** Every mutation is logged in the `provenance` array, capturing the `field`, `source`, and the `method` (e.g., `highest_confidence` or `appended_unique`).

4. Runtime Config (Projection Layer)

The pipeline treats the runtime config as a functional view layer over the canonical data.

- **Remapping:** Utilizing a path resolution helper, it reads the `from` key in the config (e.g., `emails[0]`) and projects it onto the requested `path` (e.g., `primary_email`).
- **Missing Data Handling:** Evaluates the `on_missing` rule to explicitly output `null` or omit the key entirely, degrading gracefully without throwing exceptions.
- **Separation of Concerns:** The core normalization engine is completely unaware of the projection config, satisfying the "same engine, no code changes" constraint.

5. Edge Cases & Scope

Handled Edge Cases:

1. **Missing/Malformed Sources:** If a file or API endpoint is missing (e.g., `ENOENT` errors), the extractor catches the exception, logs a warning, and returns `null`. The merge engine simply bypasses `null` records, allowing the pipeline to succeed with partial data.
2. **Malformed Formatting:** Inputting phone numbers with extensions or alphabetical characters (`98765 ext 9`) is caught by regex normalizers to prevent bad data from polluting the canonical profile.
3. **Missing Config Paths:** If the runtime config requests a path that does not exist in the candidate's data, the projection layer safely intercepts the `undefined` variable and applies the `on_missing` policy rather than crashing.

Descoped :

- **Cross-Candidate Entity Resolution:** Matching "John Doe" in Source A to "J. Doe" in Source B across thousands of files. This prototype assumes batch ingestion per singular candidate.
- **Deep Job Deduplication:** Currently, experience objects are appended. A future iteration would require a fuzzy-matching algorithm on `company_name` + `start_date` to merge identical roles.